

Software Development Lifecycle Policy

Version: v1.0 ADOPTED **Adopted:** 2026-07-21 by Sami Ben Chaalia (Security Officer) **Date:** 2026-07-21 **Owner:** Security Officer (Sami Ben Chaalia) **Framework mapping:** SOC 2 CC7.1 (System changes designed with security) · CC8.1 (Change management) · NIST SSDF (Secure Software Development Framework) · OWASP ASVS L1 baseline **Companion documents:** Change Management Policy · Access Control Policy · Confidentiality Policy · Acceptable Use Policy.

1. Purpose

Codify how RailCall code is actually written, reviewed, tested, released, and maintained. Reflects the AI-assisted workflow (Claude Code + human review) that produces most of the codebase in 2026, because ignoring that would make the policy fiction.

2. Scope

Every code repository owned by the RailCall project: - `railcall-core` (public — CLI, gateway, install scripts, compliance) - `railcall-engine` (sealed — local runtime, sealed IP boundary per `project_railcall.md`) - `railcall-contrib` (public — website, docs, mirror) - Any future repository under `github.com/patl4588`.

3. Development principles

- **Code with intent.** Every file, function, and commit exists because there's an invariant or gap it addresses. If it can't be named, it doesn't ship.
- **Read before write.** New code is written against knowledge of the existing code, not against guesses. AI-authored code is verified against the file it touches, not against the model's guess of what the file contains.
- **Tests are the invariant record.** Every test asserts a property we care about surviving. Tests are the primary artifact — code is what makes tests pass.
- **Small, reversible commits.** A commit does one thing, cites its evidence, and is safely revertible. Multi-purpose commits are refactored before push.
- **Honest naming.** Function names, variable names, module names describe what the thing does, not what we wish it did. `signing_seed_status()` returns the seed's rest posture; it does not return the seed.

4. The workflow

For most changes:

1. **Frame the change.** State the invariant being defended or the gap being closed in one sentence. If Claude Code is authoring, this is the prompt.
2. **Read the touchpoint files.** Not a summary — the actual bytes on disk today. AI agents that skip this step regularly produce plausible-but-wrong output.
3. **Write the change + at least one test.** Test must fail pre-change and pass post-change. This is what makes it a test rather than an assertion of "the code I just wrote does what I just wrote."
4. **Run the local suite** for anything the change could plausibly affect (not just the file touched — check callers).
5. **Commit with an evidence-forward message** — WHY, not just WHAT. Cite tests + affected surfaces.
6. **Push.** Per Change Management Policy §4, this triggers deploy for `railcall-core:main`.
7. **Verify post-deploy** for any Enhancement or Breaking change.

For AI-assisted commits specifically:

- **The human reviewer is Sami.** Not "the AI reviewed itself"; not "the tests passed so we're done." A human eye looks at every AI-generated commit before push (unless it falls under the standing merge authorization + is Standard-class per Change Management §6).
- **The AI cites evidence in commit messages** — file:line references, test names, invariants defended. Prose without citation reads as marketing and gets rewritten.
- **Suspected hallucinations get verified.** If Claude claims a function exists, grep for it before believing.

5. Secure coding standards

Enforced by convention + code review + the specific technical controls listed below. This is not an exhaustive OWASP list; it's the set that has actually mattered in this codebase.

5.1 Never log secrets

- No API keys, seeds, tokens, or passwords appear in any log statement.
- Vault operations log key NAMES + operation type, never values.
- Audit chain scrubs `email` and any field on the credentials + PHI lists (`P0-4` remediation).
- Every third-party call that could echo a secret in an error is wrapped to catch + redact.

5.2 Fail closed

- Every gate refuses on unknown input. `/v1/seat/checkin` returns 401 on unknown hash; issuer mint returns 503 if seed unset; entitlement verify returns free-tier on any tamper.
- Absence of configuration = refusal, never permission. `RAILCALL_ISSUER_SEED` unset means 503, not "sign anyway with a random seed."

5.3 Idempotency on external effects

- Every write that COULD be retried (webhook, meter, checkout provision) is dedup'd on a scoped nonce. Duplicate delivery is impossible in practice + provable in test.

5.4 Input validation before DB touch

- Every endpoint validates shape + bounds before opening a DB connection. 400 for malformed, 401/403 for unauthorized, 503 for unconfigured. 500 is a genuine internal error only.

5.5 No SQL string interpolation

- Every query goes through parameterized `?` placeholders + `ph()` helper for Postgres/SQLite portability. No f-string SQL. Ever.

5.6 Cryptographic algorithms per Encryption Policy §3

- Only algorithms listed in Encryption Policy §3 are used. Adding a new one requires updating that policy first.

5.7 No hardcoded secrets in git

- `.gitignore` excludes `.env`, `keys.local.json`, `*seed*.txt`, `*key*.pem`.
- Release script's leak gate refuses tarballs containing recognizable secret shapes.
- Pre-commit review catches accidents.
- If something slips: rotate immediately (Incident Response §7), then post-mortem the process failure.

5.8 Explicit typing on public surfaces

- FastAPI endpoint parameters typed with `Form(...)` / `Body(...)` etc. Type mismatch = 422 automatic.
- Python type hints on public-API functions; internal helpers may skip.

6. Test discipline

- **Unit tests** for logic; integration tests for cross-module flows; e2e tests for the full user path.
- **Named suites** per surface: `test_seat_checkin.py`, `test_stripe_lifecycle.py`, `test_activate_e2e.py`, `test_paid_full_e2e.py`, `test_cli_seed_store.py`, `test_licensing_endpoints.py`, `test_stripe_seat_wiring.py`, `test_entitlement.py` (engine), etc.
- **Every PO remediation has a dedicated test file.** `phi_guard.py` → `test_phi_guard.py` (29 checks). `audit_chain.py` → `test_audit_chain.py` (17). `seed_store.py` → `test_seed_store.py` (16) + `test_cli_seed_store.py` (8).
- **Tests assert LIMITS as well as wins.** `test_audit_chain.py:4a` asserts tail-truncation is undetectable — nobody can accidentally over-claim later.
- **Tests run against real behavior** — `test_paid_full_e2e.py` spawns a real gateway on loopback and drives the real CLI code path, no mocks on either side of the wire.
- **Fake-green is caught.** Test suites report skipped tests loudly, not silently. A silently-skipped critical assertion is worse than a red test.

7. Dependency management

- Python dependencies pinned in `requirements.txt` (railcall-core). Engine dependencies are minimal + stdlib-first by convention.
- Adding a dependency requires: 1. Justification (why we need this vs. writing it). 2. License compatibility check. 3. Security posture check (maintained? recent CVEs? source-available?). 4. Vendor Management Policy §4 onboarding if the dep is a network service (unlike a Python library).
- Removing a dependency is preferable to updating a compromised one when the removal is feasible.

8. Vulnerability response

- **Dependency CVE reported:** if the package is in our path, pin the fixed version + re-run every test suite + retrospect any station releases that might have shipped the vulnerable version (Incident Response §7.7).
- **In-house vulnerability reported:** by external researcher or internal review — treat per Incident Response §3 severity classification. `security@railcall.ai` is the intake channel (documented in `railcall-contrib/website-v2/app/legal/security`).

9. Sealed engine boundary

Per `project_railcall.md` (memory), `railcall-engine` is the sealed-IP repo. Never surface engine internals in:

- Public repos (`railcall-core` , `railcall-contrib`).
- Public station releases (the release script's leak gate enforces this — refuses tarballs containing engine-specific factory/moat artifacts like `workflow_library/` , `combinatorics_index.json` , `architecture_*` , etc.).
- Third-party pushes (never PyPI-publish engine internals; see MCP.md deliberate decision to CLI-command MCP instead of PyPI-package it, in `MCP.md` and `project-paid-tier` memory).

10. Code review

- **Sami reviews Claude-authored commits.** For Standard-class changes under the standing merge authorization, review may be post-hoc. For Enhancement + Breaking, review is pre-push.
- **Human-authored commits** get the same review, mutually. In a two-person team, review is often synchronous over Signal.
- **Review focus:** correctness > readability > style. If it's incorrect but pretty, it doesn't ship.

11. Documentation

- **Code:** minimal comments; identifiers describe what the code does. Comments are for WHY it's non-obvious (hidden constraint, subtle invariant, workaround, surprising behavior).
- **Public APIs:** documented on the endpoint or CLI command itself, not in a separate file that drifts.
- **Compliance:** this policy set + the SRA + the BAA draft. See `compliance/policies/README.md` for the index.

12. Third-party service integrations (BYOK)

When we add a new integration to the station (Slack, Airtable, GitHub, etc.):

1. Design + implement locally with the customer's own credential + no data leaving their box.
2. Add to the registry (`integration_registry.py`) with proper `action_class` per the effect-node classification (see canvas + workflow_engine docs).

3. PHI-guard the payload path — every integration must go through `approval_airlock.py:redact()`.
4. Test with a fixture + a real cred (dev-scoped).
5. Update `Vendor Management Policy §3` only if we ourselves rely on the vendor for hosted service — a BYOK integration is customer-scoped and doesn't create a subprocessor relationship.

13. Release + deploy

Per Change Management Policy §5 (station release ceremony). This policy doesn't repeat the steps; it points at them.

Deploys of the gateway happen automatically on push to `railcall-core:main`. Not automatic: station releases (require the full ceremony) or VPS website rebuilds (require `pm2 restart` after a `public/` change).

14. Review

At each station release + at least annually. Any new secure-coding rule added to §5 requires an example of the class of failure it prevents, so future-us knows why it exists.

15. Related documents

- **Change Management Policy** §4, §5 — the deploy pipeline this SDLC feeds.
- **Access Control Policy** §6 — the credential surface development touches.
- **Confidentiality Policy** §5 — the "what we do NOT hold" invariants coding is bound to preserve.
- **Acceptable Use Policy** §3 — the cultural rules (no fake-green, evidence before prose) that this policy operationalizes.
- **Incident Response Policy** §7.7 — the dependency-compromise runbook §8 refers to.